

File Edit Selection View Go Run Terminal Help

langraph

lgraph_quickstart.ipynb X

langgraph1.ipynb

langgraph2.ipynb

Llama_3_1_8B.ipynb

lgraph_quickstart.ipynb

empty cell

Generate + code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.10.11

OPEN EDITORS 1 unsaved

lgraph_quickstart.ipynb

langgraph1.ipynb

langgraph2.ipynb

Llama_3_1_8B.ipynb

LANGGRAPH

langgraph1.ipynb

langgraph2.ipynb

lgraph_quickstart.ipynb

Llama_3_1_8B.ipynb

[7]

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

from IPython.display import Image, display
```

Python

>

```
from typing import TypedDict
from langgraph.graph import StateGraph

class MyState(TypedDict):
    message: str

    (class) MyState

def start_node(state: MyState) -> MyState:
    print("Start node received:", state["message"])
    return {"message": state["message"] + " -> processed by start"}

def end_node(state: MyState) -> MyState:
    print("End node received:", state["message"])
    return state

# Create graph builder
builder = StateGraph(MyState)
```

[11]

Python

> OUTLINE

> TIMELINE

[12]

```
# Add nodes
```

Python

Ln 5, Col 1 Spaces: 4 CRLF Cell 7 of 7 Go Live

File Edit Selection View Go Run Terminal Help

langraph

EXPLORER

OPEN EDITORS 1 unsaved

langraph_quickstart.ipynb

langgraph1.ipynb

langgraph2.ipynb

Llama_3_1_8B.ipynb

LANGGRAPH

langgraph1.ipynb

langgraph2.ipynb

lgraph_quickstart.ipynb

Llama_3_1_8B.ipynb

lgraph_quickstart.ipynb X

langgraph1.ipynb

langgraph2.ipynb

Llama_3_1_8B.ipynb

lgraph_quickstart.ipynb > empty cell

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline

Python 3.10.11

```
builder.add_node("start", start_node)
builder.add_node("end", end_node)
```

[12] <langgraph.graph.state.StateGraph at 0x1dd895bf6a0>

```
# Define workflow
builder.set_entry_point("start")
builder.add_edge("start", "end")
```

[13] <langgraph.graph.state.StateGraph at 0x1dd895bf6a0>

```
# Compile graph
app = builder.compile()
```

[14]

```
# Run the graph
result = app.invoke({"message": "Hello"})
print("Final Output:", result)
```

[15]

```
Start node received: Hello
End node received: Hello -> processed by start
Final Output: {'message': 'Hello -> processed by start'}
```

OUTLINE

TIMELINE

Ln 5, Col 1 Spaces: 4 CRLF Cell 7 of 7 Go Live